

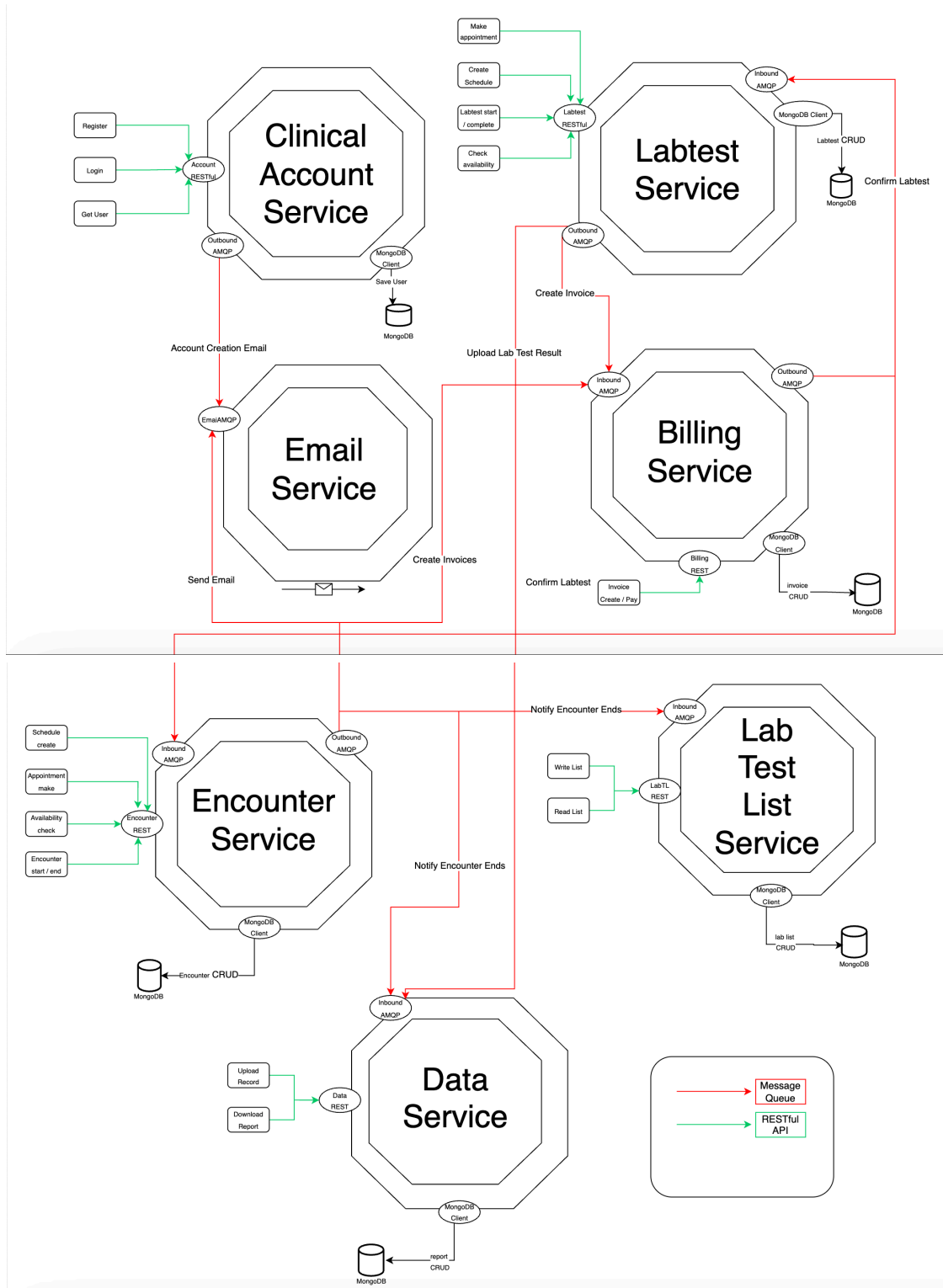
Phase #4: Architecture Report

Project: ClinicAI

Author: Weifan Wu, Yangyang Zhang

Date: 10/12/2025

1. High-Level Architecture



1.1 External Interfaces & Inter-service Communications

1. External REST APIs: All REST APIs in this application are public-facing.
 - a. Account (register/login/get user)
 - b. Encounter (schedules/availability/appointments/start-end)
 - c. Billing (get/pay invoice)
 - d. LabTest (schedule/book/start-end/upload result)
 - e. LabTest List (get/update tests for an encounter)
 - f. Data (upload recording, get report, data partner communication)
2. Message bus (RabbitMQ, exchange `clinica.events`; all for internal-service calls only)
 - Booking/billing:
 - `appointment.booked` & `labtest.appointment.booked` → Billing creates invoices;
 - `billing.payment.authorized` → Encounter/LabTest confirm appointments.
 - Lab + data:
 - Encounter end emits `lab.tests.update` → LabTest-List creates list;
 - Encounter emits `data.analysis.requested` → Data pre-creates report;
 - LabTest completion emits `lab.tests.result` → Data stores lab payload.
 - Notifications:
 - Encounter/LabTest confirmations emit `notification.email.*`;
 - Account creation emits `notification.email.welcome`;
 - Email service subscribes to `notification.email.#` to send messages.
 - Observability/faults: on bad/missing payloads we emit:
 - `error.billing.invoice`, `error.encounter.billing`, `error.labtest.billing`, `error.lablist.update`, `error.data.report`, `error.email.send`.

1.3 Communication Choice Justification

Justification of using RESTful API:

- In this project, all APIs are implemented using RESTful HTTPS interfaces. HTTPS is a widely adopted and standardized communication protocol in modern web systems, making it a natural choice for building stable and interoperable services. For example, the frontend browser must send HTTPS requests for user registration and login. These operations require immediate feedback to the user, such as whether the registration succeeded or whether the provided credentials are correct. Therefore, implementing these functionalities using synchronous RESTful APIs is appropriate because it guarantees a direct request–response cycle.
- Another benefit of using REST is that it follows a **stateless communication model**. The server does not store session information about the client, which simplifies scalability. Stateless APIs allow multiple instances of the same service to be deployed behind a load balancer without worrying about shared session data. As a result, the system can be horizontally scaled more easily and cost-effectively.

Justification of Using RabbitMQ

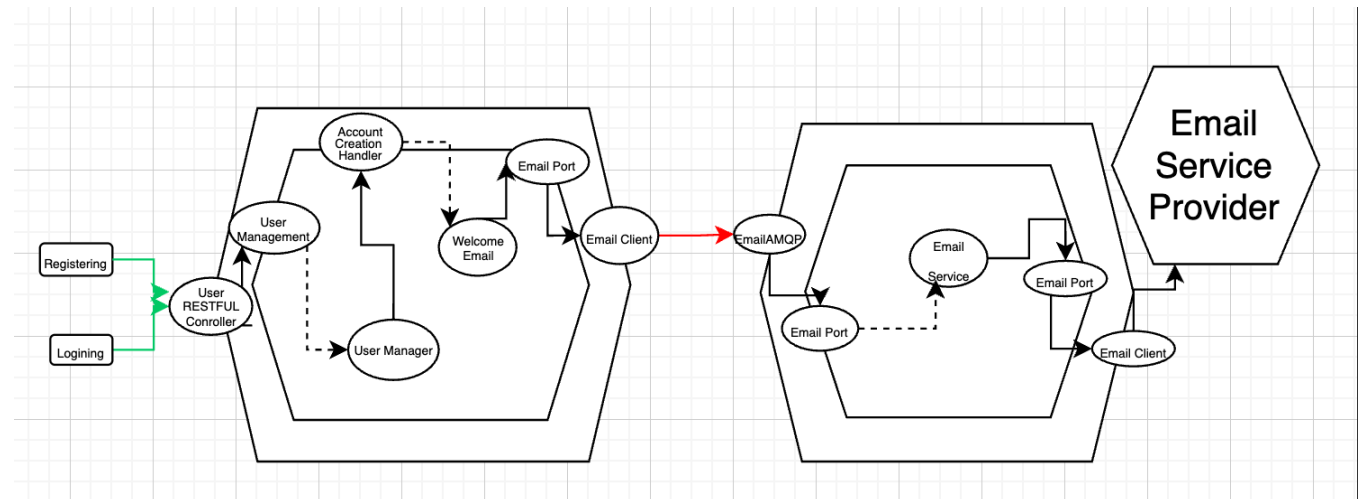
- RabbitMQ is mainly used in this application to **decouple services** and improve system reliability. For example, the account service and email service communicate via RabbitMQ instead of direct HTTP calls. This ensures that if the email service becomes unavailable or encounters errors, the account service and client-facing application can still function normally. This decoupled design improves fault tolerance and avoids cascading failures.
- In addition, RabbitMQ enables easier extensibility. Loosely coupled services can evolve independently—adding or modifying one service does not require changes in other components. This design principle is also applied to the latest service and billing service, where RabbitMQ ensures asynchronous communication with better resilience and modularity.
- Another important design decision is that the labtest service sends test reports to the data service through RabbitMQ events instead of storing the report in a database first and then notifying the data service to retrieve it. Since the test reports are lightweight, directly publishing them as messages reduces latency

and simplifies the communication flow. This approach takes full advantage of RabbitMQ's low-latency message handling and avoids unnecessary database writes when not needed.

2. Service Decomposition

In the subsections below, we outline each service's responsibility, why it matters in the flow, and how it fits our hexagonal (ports/adapters) design.

2.1 Account and Email Service



2.1.1 Account Service

2.1.1.2 Responsibilities

- Handle user registration for Patients, Clinicians, and Lab Technicians via REST endpoint, validating uniqueness of email and generating unique account IDs.
- Expose REST APIs to authenticate users (login) and retrieve account information by ID.
- Triggers welcome email notifications upon successful registration by calling the email service port.
- Persist all account data (credentials, roles, profile information) in the accounts repository; keep authentication and user management behind ports/adapters (DB, Email).

2.1.1.3 Relevance with Proposal

- In the typical ClinicAI scenario, every user (Patient, Clinician, Lab Technician) must have a verified account before performing any role-specific actions. The Account service enables this by providing registration and authentication as the entry point to the system.
- By this design, we establish identity and access control at the beginning of every workflow (register → login → perform actions). All downstream services (Encounter, LabTest, Billing) rely on authenticated user identities to associate appointments, tests, and payments with the correct users. The welcome email integration ensures users receive immediate confirmation of their registration, improving user experience and traceability.
- Using a centralized account service keeps user management loosely coupled from other business domains, so changes to authentication mechanisms (e.g., adding OAuth, 2FA) don't impact encounter or billing logic.

2.1.1.4 Hexagonal Design

- Clear ports: inbound REST ([AccountController](#) for registration/login/retrieval), outbound ports for account persistence ([AccountRepositoryPort](#)) and email notifications ([EmailServicePort](#)).

- Adapters swap without touching domain: [AccountRepository](#) adapter for in-memory storage (can be replaced with MongoDB via [AccountMongoRepository](#)), [EmailServiceImpl](#) adapter for email notifications (currently mocked, can be replaced with SendGrid/AWS SES); any can be swapped without changing domain logic.
- Domain rules stay in core services: [AccountServiceImpl](#) contains all business logic (email validation, duplicate checking, password verification), independent of REST, database implementation, or messaging transport.
- Error Handling: Implement validation for duplicate email registrations and invalid login credentials; publish `error.account.*` events to RabbitMQ when registration/login failures occur due to bad/missing data (e.g., missing email, duplicate accounts, authentication failures).

2.1.2 Email Service

2.1.2.1 Responsibilities

- Consume email notification requests from AMQP queue ([email.queue](#)) via RabbitMQ listener when other services need to send emails (e.g., welcome emails, appointment confirmations, lab results).
- Process incoming email payloads containing recipient ([to](#)), subject, and body fields.
- **Error Handling & Publishing:** Validate incoming email payloads (required fields: [to](#), [subject](#), [body](#)); publish `error.email.send` events to RabbitMQ when email processing fails due to missing data, malformed payloads, or delivery failures, enabling centralized error monitoring and retry mechanisms.

2.1.2.2 Relevance with Proposal

In the typical ClinicAI workflow, timely notifications are critical for user engagement and system transparency. The Email service makes this viable by providing asynchronous, reliable email delivery triggered by key events across the system:

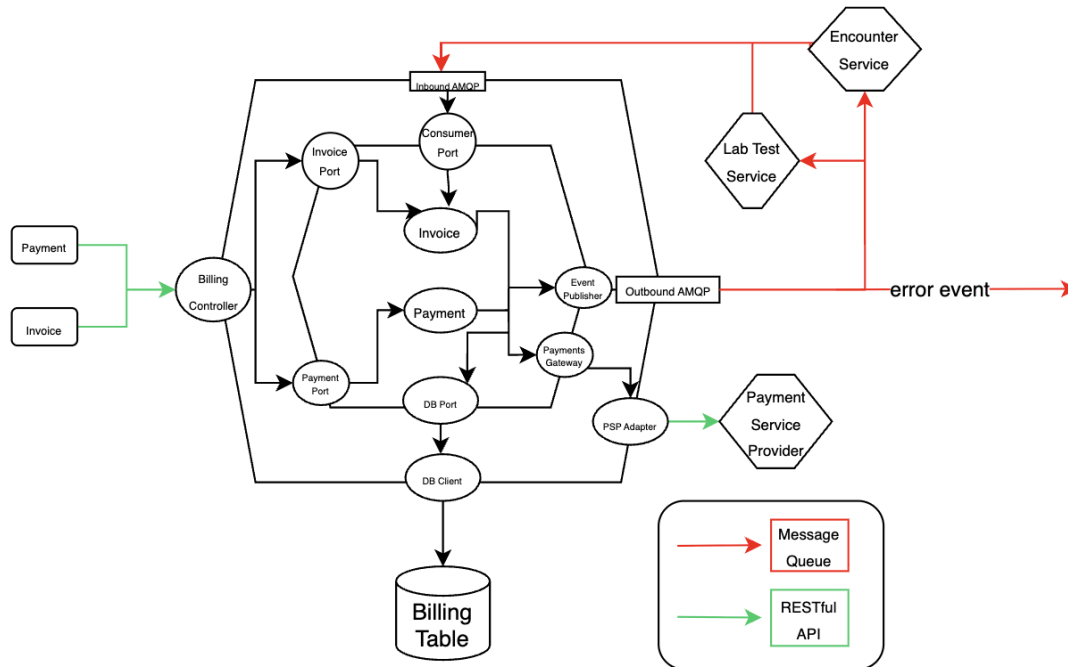
- Welcome emails when accounts are registered
- Appointment confirmation emails after successful booking and payment
- Lab result notifications when tests are completed
- Billing receipts after payment authorization

2.1.2.3 Hexagonal Design

- Clear ports: inbound AMQP consumer ([EmailConsumerAdapter](#) with `@RabbitListener` on [email.queue](#)), outbound port for email delivery logic ([EmailServicePort](#)).
- Domain rules stay in core services: [EmailServiceUseCase](#) contains all email processing logic (payload parsing, validation, delivery orchestration), independent of message transport or email provider implementation

2.2 Billing Service

2.2.1 Service Hexagonal Architecture



2.2.2 Responsibilities

- Create invoices when `appointment.booked` or `labtest.appointment.booked` events arrive (AMQP consumer).
- Expose REST to fetch/pay invoices; on payment, charge via PSP adapter and mark invoice paid in Mongo.
- Publish `billing.payment.authorized` so Encounter/LabTest can confirm appointments; emit `error.billing.*` on bad/missing data.
- Persist all billing state in invoices collection; keep payment orchestration behind ports/adapters (DB, PSP, AMQP).

2.2.3 Relevance with Proposal

In the typical scenario, every encounter and lab test in our flow must be charged, authorized, and auditable before downstream steps proceed. Billing service makes this viable by issuing invoices as soon as appointments are booked, taking payments (encounter and lab fees), and only then emitting `billing.payment.authorized` so Encounter/LabTest can confirm.

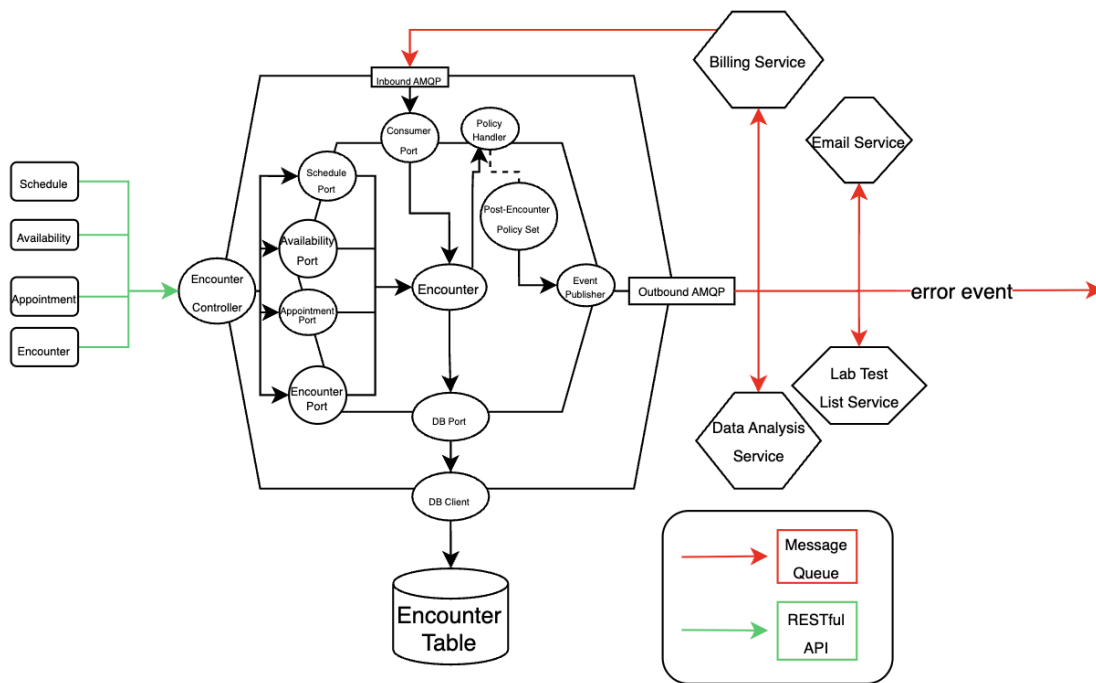
By this design, we follow the intended workflow (book → pay → confirm). That way, all later steps (encounters, lab tests, reports) only run for paid/traceable work. Using AMQP keeps billing loosely coupled, so other services don't break if billing is slow or temporarily down.

2.2.4 Hexagonal Design

- Clear ports: inbound REST (BillingController), inbound AMQP consumer (BillingConsumers), outbound ports for invoices/payments persistence and PSP, outbound event publisher.
- Adapters swap without touching domain: Mongo adapter for invoices, PSP adapter for payments, Rabbit publisher for events; any can be replaced (e.g., different DB/PSP) with no domain change.
- Domain rules stay in core services (InvoiceService/PaymentService), independent of transport, DB, or message queue.

2.3 Encounter Service

2.3.1 Service Hexagonal Architecture



2.3.2 Responsibilities

- Handle schedules, availability, booking, and lifecycle (start/end) of encounters.
- Persist appointments/encounters in Mongo; enforce status transitions (book → confirm on billing, start → end).
- Emit/consume events: consume billing.payment.authorized to confirm; publish appointment.booked/confirmed, lab.tests.update, data.analysis.requested, and notification.email.encounter.confirmed; emit error.encounter.* on bad payloads or invalid transitions.
- Drive downstream flows: triggers lab test list creation, pre-creates data reports, and notifies email service.

2.3.3 Relevance with Proposal

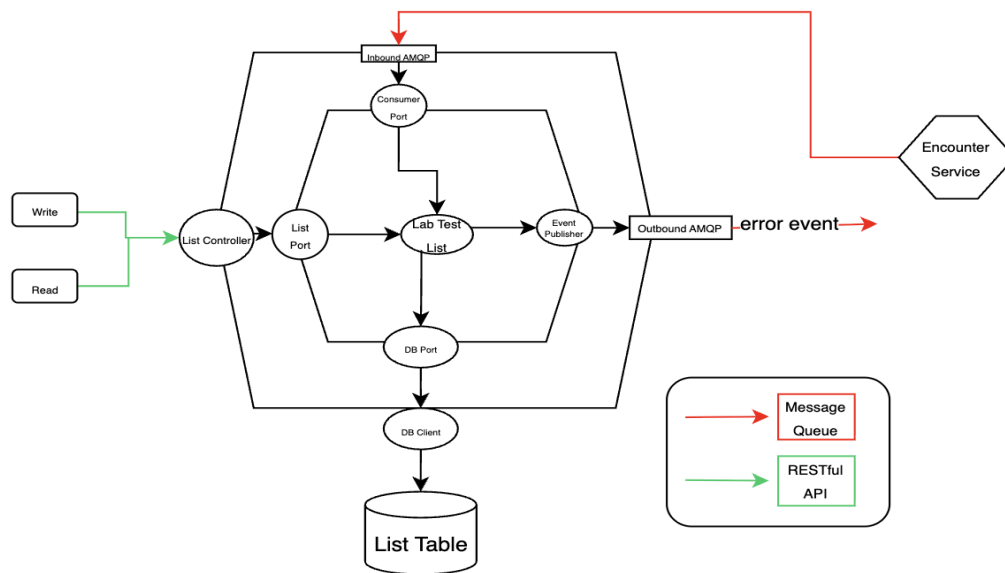
This service keeps the patient-clinician visit aligned with the event-storm: schedule → book → pay → confirm → conduct encounter → produce lab list and data for the smart report. It's the core orchestrator that binds billing, lab test list, data, and email around a single visit.

2.3.4 Hexagonal Design

- Ports for schedules, availability, appointments, encounters; AMQP consumer/publisher ports; DB store ports.
- Adapters can swap independently (Mongo repo, Rabbit publisher/consumer); domain services/policies stay untouched.
- Policies (`EventDrivenEncounterPolicyHandler`) capture cross-service reactions (publish lab/data/email events) without coupling to transport or persistence.

2.4 Lab Test List Service

2.4.1 Service Hexagonal Architecture



2.4.2 Responsibilities

- Consume `lab.tests.update` (from Encounter) to create a lab list for the encounter.
- Expose REST to read/update the lab test list for that encounter (patient/clinician-facing).
- Persist lists in Mongo (`lab_lists`), emit `error.lablist.update` on missing IDs or failed create.

2.4.3 Relevance with Proposal

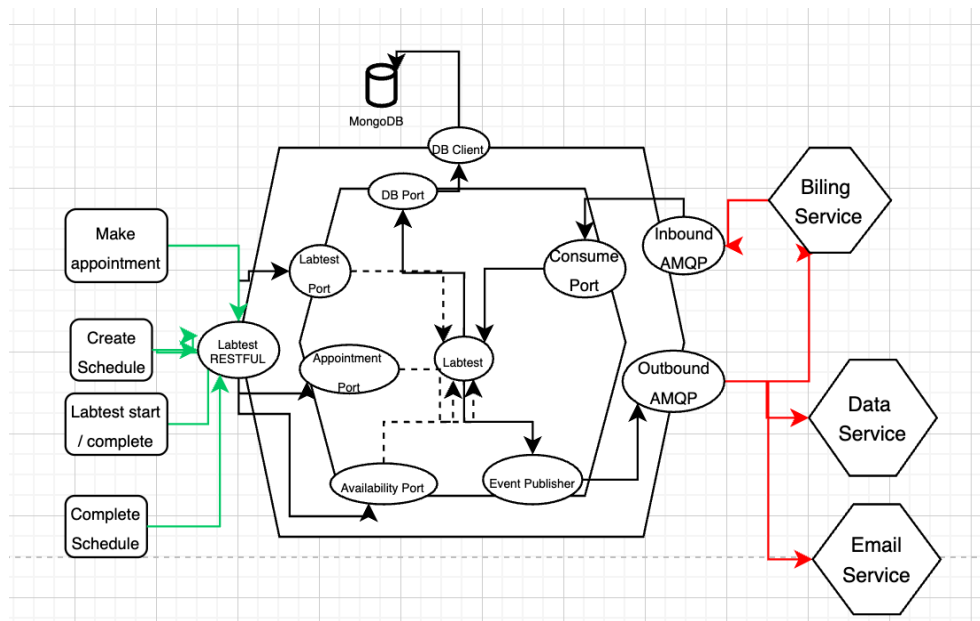
This service is basically the bridge between the clinician's encounter and the downstream lab test: ensures every encounter produces an editable, traceable lab list before a corresponding lab test appointment is booked or performed.

2.4.4 Hexagonal Design

- Clear ports: inbound AMQP consumer for `lab.tests.update`, inbound REST controller for list check/modification; outbound Mongo adapter for persistence; Rabbit publisher for errors.
- Domain logic (create/update tests) isolated from transport/storage, so adapters can change without touching the core.

2.5 Labtest Service

2.5.1 Service Hexagonal Architecture



2.5.2 Responsibilities

- Appointment Management for Lab Tests: Provide REST APIs to book lab test appointments with available lab-technician time slots, retrieve appointment details, and confirm appointments after payment.
- Lab Test Lifecycle Management: Provide REST APIs to manage the full lifecycle of lab tests, transitioning through the following states:
 - PENDING_PAYMENT → CONFIRMED → IN_PROGRESS → COMPLETED.
- Lab Result Events: Publish lab-test-result events when tests are completed, including result so the Data service can process and store results for patient medical records.
- Error Handling & Event Publishing: Validate all billing events and appointment operations; publish error events when confirmation fails due to missing data, expired payment windows, or invalid states, enabling centralized error tracking and recovery mechanisms.

2.5.3 Relevance with Proposal

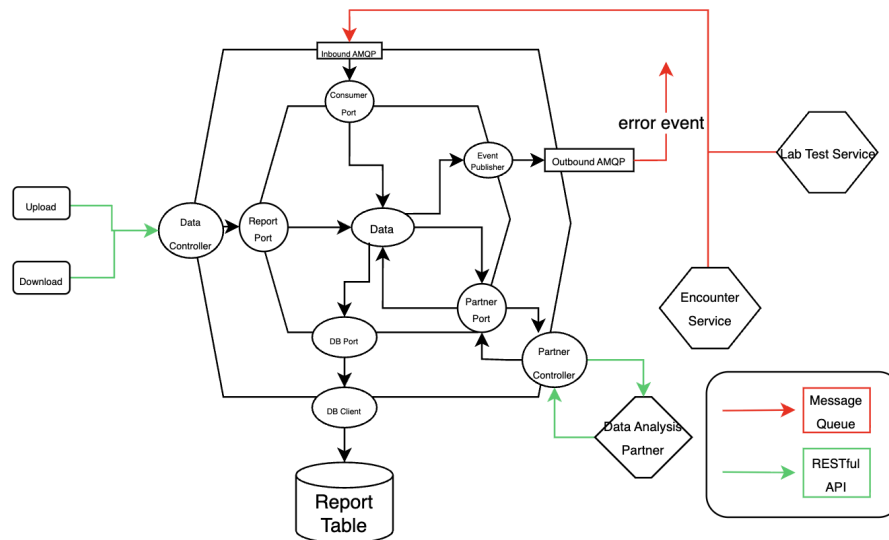
- The LabTest service directly supports the proposal's core objective: enabling a reliable, auditable, and scalable workflow for diagnostic testing. It ensures that lab tests cannot proceed without a valid appointment, confirmed payment, and clearly assigned resources. By enforcing this end-to-end workflow and coordinating asynchronously with surrounding services, the system maintains operational integrity even under high load or temporary service disruptions.
- The service also aligns with the proposal's event-driven architecture. Lab test confirmations, completions, and other key state changes are propagated as domain events, enabling Billing, Notification, and Data services to react independently without tight coupling. This design enhances extensibility and ensures that future services—such as analytics or patient-facing features—can be added without modifying core LabTest logic.

2.5.4 Hexagonal Design

- Inbound REST (AppointmentController, LabTestController): Expose appointment booking/confirmation and lab test start/end endpoints.
- Inbound AMQP consumer (BillingEventConsumer): Listens for payment-authorized events to confirm appointments.
- Outbound event publisher (EventPublisher): Publishes events for appointment booked, test started, test completed, and error conditions.
- Outbound persistence ports: Store lab test data; abstract appointment and account data storage.

2.6 Data Service

2.6.1 Service Hexagonal Architecture



2.6.2 Responsibilities

- Consume `data.analysis.requested` to pre-create a report for the encounter; consume `lab.tests.result` to attach lab payload.
- Expose REST to upload encounter recording metadata and lab payload, and to download the report.
- Persist reports in Mongo (reports); auto-mark ready when both recording + lab payload exist; emit `error.data.report` on missing fields or unknown encounter.
- Interact with the data analysis partner for data processing and report generation.

2.6.3 Relevance with Proposal

It is the “smart report” hub: collects encounter recording and lab results, packages them, and exposes the final report to the patient/clinician—core to the ClinicAI promise.

2.6.4 Hexagonal Design

- Clear ports: inbound AMQP consumer for encounter/lab events; inbound REST controller for uploads/downloads; inbound partner callback controller for receiving reports; Mongo adapter for persistence; Rabbit publisher for errors; outbound partner client for sending collected encounter and lab test data.
- Domain logic stays isolated from transport/storage; adapters swap without changing the core.

3. Platform Trade-Off Analysis

3.1 In/Out of Scope Services

- **In-scope:** full path from account → encounter → billing → lab test list/lab test → data report (recording + lab payload) → email notifications; Mongo persistence; RabbitMQ events; containerized demo.
- **Out-of-scope:** real AI analysis partner, rich email templates, retries/compensation flows, granular auth/RBAC, multi-PSP support.

3.2 Design Trade-Offs

- Single Mongo for all services vs. per-service DB: chose one DB for demo simplicity and unified debugging; sacrifices stronger isolation.
- RabbitMQ for cross-service flows vs. synchronous RPC: chose async for decoupling and resilience; adds eventual consistency and ordering complexity.
- Stubbed partner call vs. real external ML: kept flow deterministic for demo; defers integration risk.
- One exchange (clinical.events) for all events vs. multiple exchanges: simpler wiring; requires careful routing keys to avoid noise.
- Simplified emails (plain payload) vs. templating/delivery guarantees: faster delivery; less rich UX and no guaranteed delivery/reporting.

3.3 Consequences

- Faster iteration and clearer observability (all state in one DB, single exchange), but tighter resource coupling and weaker fault isolation.
- Eventual consistency is acceptable for demo, but would need idempotency/retries for production.
- Partner/PSP/email depth deferred—good for predictability now, but additional work to harden later.

4. Platform Analysis Dimensions

4.1 Observability & Fault tolerance

- Event logging at publish/consume; error events on bad/missing payloads (`error.*`).
- Decoupling via RabbitMQ prevents cascading failures; still minimal retries/DLQs—could add for production.

4.2 Scalability

- Stateless REST surfaces; can scale horizontally behind load balancer.
- Single Mongo could bottleneck; sharding or per-service DB would improve isolation.
- RabbitMQ central exchange scales, but needs tuning/quotas under load.

4.3 Maintainability & Coupling

- Ports/adapters per service; domain isolated from transport/DB.
- Shared DB and shared exchange increase operational coupling; future step: split schemas/exchanges.
- Clear routing keys and collections reduce mental load.

4.4 Testability

- Domain logic behind ports allows in-memory tests/mocks.
- REST + event flows can be exercised via the provided scripts/Postman.
- Lacks full contract/consumer-driven tests and chaos/failure-injection; could add for higher confidence.

5. Feedback Integration

We highly appreciated the feedback given during the review and acted based on it by adding fault-tolerance/observability hooks: every AMQP consumer now validates payloads and, on missing/invalid data, emits `error.*` events (`error.billing.invoice`, `error.encounter.billing`, `error.labtest.billing`, `error.lablist.update`, `error.data.report`, `error.email.send`) on the shared `clinical.events` exchange. We also log publish/consume events consistently across services.